

ACE-Omega Academic Universe

Technical Manual & User Guide

v3.1 / v4.0 Components

Field	Value
System	ACE-Omega Academic Universe (Streamlit Web Application)
Version	v3.1 (model backend) / v4.0 (components)
Framework	Streamlit (Python)
Visualization	Three.js r128 (embedded via Streamlit HTML components)
AI Backends	OpenAI API, Ollama (local), GGUF (llama-cpp), Fallback (rule-based)
Languages	English / Chinese (i18n)
Primary Input	Academic literature files (CSV, JSON, XLSX, PDF, TXT, MD, DOCX, BIB)
Primary Output	Interactive 3D celestial visualization + AI-powered analysis

This document is generated from source code audit. All conclusions are traceable to specific files, functions, and code lines.

Table of Contents

0 Executive Summary

1 Project Structure and Module Map

1.1 File Tree

1.2 Entry Point and Startup

1.3 Module Responsibility Table

1.4 Data Flow and Control Flow

2 Formula Table (Complete)

2.1 Discovery Rules

2.2 Formula List

2.3 Formula Dependencies

3 System Mechanism Diagram

4 Methodology

5 Computation Methods

5.1 Data Formats and Field Definitions

5.2 Preprocessing Steps

5.3 Core Algorithm Steps

5.4 Parameter List

5.5 Output Products

5.6 Complexity and Performance Bottlenecks

6 FAQ and Troubleshooting

7 Getting Started Guide

8 Supplementary Notes

A Appendix: Full Dependency List

B Appendix: Configuration Reference

C Appendix: Directory Structure

0. Executive Summary

What This System Is

ACE-Omega Academic Universe is a Streamlit-based web application that transforms scattered academic literature data into an interactive 3D celestial visualization. Each academic paper becomes a celestial body; citation counts determine mass and radius; disciplinary modules determine color and spatial clustering. Highly cited papers become 'black holes' or 'Dyson spheres.' An AI analysis subsystem allows natural-language querying of the loaded dataset.

Problem Solved (Input -> Process -> Output)

Input: Academic literature files in any of 15+ formats (CSV, JSON, XLSX, PDF, TXT, MD, DOCX, DOC, BIB, RIS, XML, HTML, TSV). Fields may be incomplete or inconsistently named. Process: The system parses files, normalizes column names via fuzzy matching, converts records to Paper objects, computes celestial properties (mass, radius, 3D coordinates, black hole / Dyson sphere classification), and renders an interactive Three.js 3D scene with tooltips, labels, search highlighting, module connection lines, nebula particles, and gravity grids. AI backends (OpenAI, Ollama, GGUF, or fallback) provide natural-language analysis of the dataset. Output: Interactive 3D universe visualization, dataset statistics, AI analysis results, session save/load (JSON), PNG export of the 3D canvas, CSV data export.

Core Capabilities

- Universal file parsing: 15+ academic file formats with multi-strategy extraction and graceful fallback.
- Intelligent column normalization: Fuzzy matching maps arbitrary column names to standard paper schema.
- Celestial property computation: Log-scale mass/radius, Pareto-distributed citation modeling, black hole / Dyson sphere classification.
- 3D visualization via Three.js r128: Multi-layer star fields, nebula particles, module connection lines, animated rings, gravity grids.
- Global and per-node label system: CSS2D-like overlay labels with distance-based fade.
- AI-powered analysis: Multi-backend (OpenAI, Ollama, GGUF) with automatic fallback chain.
- Bilingual i18n: Full English and Chinese interface.
- Session persistence: Save/load entire analysis sessions as JSON.
- arXiv batch import: Direct search and import from arXiv API.
- Industrial dark theme: Custom CSS with Inter + Noto Sans SC font stack.

Runtime Requirements

Python 3.8+. Core dependencies: streamlit, pandas, numpy, pyyaml. Optional: pdfplumber or pypdf (PDF parsing), python-docx (DOCX), openpyxl (XLSX), arxiv (arXiv import), ollama (Ollama backend), openai (OpenAI backend), llama-cpp-python (GGUF backend).

Confirmed Limitations (Code-Evidenced)

- No unit tests exist in the provided codebase.
- No requirements.txt or pyproject.toml for dependency version locking.
- No authentication or access control for the web interface.

- Random seed is not fixed in `generate_demo()` or `compute_properties()`; results are not bitwise reproducible.
- GGUF model is reloaded on every inference call (`infer_gguf`); no model caching.
- Connection lines in Three.js use $O(n^2)$ distance computation per module group.
- The arXiv import sets `cited_by_count=0` for all imported papers (arXiv API does not provide citation counts).
- The `pages/` directory (Universe, Analysis, Timeline) is referenced in the sidebar but not included in the provided codebase; only the Home page (`app.py`) is present.
- Error logging to `./logs/model_errors.jsonl` is declared but never written to (no code calls the log writer).

1. Project Structure and Module Map

1.1 File Tree

ACE-Omega/	
app.py	# Main entry point (Home page + sidebar)
config.yaml	# Runtime configuration (auto-generated)
utils/	
__init__.py	# Package marker
ace_model.py	# AI backend: infer, health check, backend status
data_processor.py	# Paper data: parse, compute, cluster, demo, export
file_parser.py	# Universal file parser (15+ formats)
i18n.py	# Bilingual string table (EN/ZH)
theme.py	# CSS theme injection
components.py	# Three.js boilerplate, UI widgets, session I/O
pages/	# [NOT PROVIDED] Streamlit multi-page directory
1_Universe.py	# [INFERRED] 3D universe visualization page
2_Analysis.py	# [INFERRED] AI analysis page
3_Timeline.py	# [INFERRED] Timeline replay page
4_Settings.py	# [INFERRED] Settings page
models/	# GGUF model storage (created at runtime)
logs/	# Error log directory (created at runtime)

Items marked [NOT PROVIDED] or [INFERRED] are referenced in code but absent from the supplied files.

1.2 Entry Point and Startup

Entry file: app.py. Startup command:

```
streamlit run app.py
```

app.py calls st.set_page_config() at line 21, inject_theme() at line 22, and initializes session state defaults at lines 41-48. The sidebar is built at lines 53-191. The home page content (upload, demo, stats, arXiv import) begins at line 196. Streamlit's native multi-page support expects additional pages in a pages/ subdirectory; these files are not provided.

1.3 Module Responsibility Table

Module / File	Responsibility	Key Functions / Classes	Called By
app.py	Application entry, sidebar, Home page logic	load_page_config(), save_config(), arxiv_import_logic	Streamlit runtime
utils/ace_model.py	AI inference with fallback chain (ollama, openai, gpt4o, etc.)	infer(), health_check(), check_model_packages(), ollama_pull_model()	app.py, ollama_utils.py
utils/data_processor.py	Paper data modeling, celestial Paper (data class), add to database, generate statistics	Paper(), add_paper(), generate_statistics(), papers_to_json()	app.py, components.py
utils/file_parser.py	Universal file parsing (15+ formats)	parse_file(), normalize_dataframe(), parse_csv/jpg/png/pdf/docx/bib/markdown(), _text_t	app.py
utils/i18n.py	Bilingual string lookup (EN/ZH)	key, lang), STRINGS dict	app.py, components.py
utils/theme.py	CSS theme injection for Streamlit	inject_theme(), THEME_CSS string	app.py
utils/components.py	Three.js HTML/JS generation (Arxiv visualization, session boilerplate, etc.)	highlight_playseth(), orbit_controls(), th	app.py

1.4 Data Flow and Control Flow

1) User uploads file(s) via Streamlit file_uploader or clicks 'Load Demo' or enters an arXiv query. 2) File bytes are dispatched to file_parser.parse_file(), which selects a format-specific parser by extension. 3) The resulting DataFrame is passed to normalize_dataframe(), which fuzzy-matches columns to a standard schema (title, authors, year, cited_by_count, abstract, keywords, doi, module, journal). 4) The normalized DataFrame is converted to Paper objects via df_to_papers(). 5) compute_properties() assigns mass, radius, 3D coordinates (x, y, z), black hole / Dyson sphere flags, and pagerank. 6) compute_stats() produces summary statistics stored in st.session_state. 7) The Home page displays metrics and a top-papers table. 8) The Universe page (not provided) renders papers as Three.js spheres using components.py helper functions. 9) The AI Analysis widget sends dataset context + user query to ace_model.infer(), which tries backends in a fallback chain. 10) Session state can be serialized to JSON and deserialized back.

2. Formula Table

2.1 Formula Discovery Rules

All formulas below are extracted from explicit code expressions in the provided Python files. No library-implicit formulas (PCA, TF-IDF, etc.) are used in this codebase. The system uses elementary mathematical operations: logarithms, linear mappings, Pareto distribution sampling, Gaussian noise, and threshold comparisons.

2.2 Formula List

F-001: Paper Mass

Field	Detail
Formula	$mass = 1.0 + \log_{10}(1 + cited_by_count + 1) * 2.5$
Location	data_processor.py : compute_properties(), line ~78
Variables	cited_by_count: integer ≥ 0 , count of citations received by the paper.
Purpose	Determines visual mass of the celestial body. Higher cited papers have larger mass. Mass is not used for gravitation.
Numerical Stability	$\log_{10}(1 + c + 1)$ is always > 0 for $c \geq 0$. Minimum mass = $1.0 + \log_{10}(2) * 2.5 = 1.75$. No division-by-zero risk.
Complexity	$O(1)$ per paper, $O(n)$ total. Called once per dataset load.

F-002: Paper Radius

Field	Detail
Formula	$radius = 3.0 + \log_{10}(1 + cited_by_count + 1) * 2.5$
Location	data_processor.py : compute_properties(), line ~79
Variables	cited_by_count: integer ≥ 0 .
Purpose	Determines visual size of the sphere in the 3D scene. Minimum radius = $3.0 + \log_{10}(2) * 2.5 = 3.75$.
Numerical Stability	Same as F-001. Always positive.
Complexity	$O(1)$ per paper.

F-003: Black Hole Detection (Citation Threshold)

Field	Detail
Formula	$is_blackhole = True$ if $cited_by_count > mean_cited * 50$
Location	data_processor.py : compute_properties(), line ~81
Variables	mean_cited: arithmetic mean of all papers' cited_by_count. 50: hardcoded multiplier.
Purpose	First pass of black hole classification. Papers with citations exceeding 50x the mean are flagged.
Numerical Stability	mean_cited guarded by conditional ($mean_cited > 0$).
Complexity	$O(1)$ per paper, requires $O(n)$ mean calculation.

F-004: Black Hole Detection (Top 0.1%)

Field	Detail
Formula	$\text{is_blackhole} = \text{True if rank_index} < \max(1, \text{floor}(n * 0.001))$
Location	data_processor.py : compute_properties(), lines ~84-88
Variables	n: total number of papers. rank_index: position after sorting by cited_by_count descending.
Purpose	Second pass: the top 0.1% of papers by citation count are always classified as black holes.
Numerical Stability	$\max(1, \dots)$ ensures at least 1 paper qualifies for any non-empty dataset.
Complexity	$O(n \log n)$ due to sorting.

F-005: Dyson Sphere Detection

Field	Detail
Formula	$\text{is_dyson} = \text{True if rank_index} < \max(1, \text{floor}(n * 0.01)) \text{ AND cited_by_count} > \text{mean_cited} * 10$
Location	data_processor.py : compute_properties(), lines ~89-91
Variables	n: total papers. mean_cited: mean citation count. 10: hardcoded multiplier.
Purpose	Top 1% of papers that also exceed 10x mean citations are flagged as Dyson spheres.
Numerical Stability	Same guards as F-003 and F-004.
Complexity	$O(1)$ per paper after sort.

F-006: PageRank (Simplified)

Field	Detail
Formula	$\text{pagerank} = 1.0 - (\text{rank_index} / \max(n, 1))$
Location	data_processor.py : compute_properties(), line ~87
Variables	rank_index: 0-based position in descending citation sort. n: total papers.
Purpose	Assigns a linear rank score [0, 1] based on citation order. This is NOT true PageRank; it is a linear rank normalization.
Numerical Stability	$\max(n, 1)$ prevents division by zero.
Complexity	$O(1)$ per paper.

F-007: X-coordinate (Time Axis)

Field	Detail
Formula	$x = ((\text{year} - \text{year_min}) / \text{year_span} - 0.5) * 80 + N(0, 2)$
Location	data_processor.py : compute_properties(), line ~98
Variables	year: publication year. year_min: earliest year. year_span: $\max(\text{year_max} - \text{year_min}, 1)$. $N(0,2)$: Gaussian noise, sig
Purpose	Maps publication year to x-coordinate, spreading papers along a time axis with jitter.
Numerical Stability	year_span guarded by $\max(\dots, 1)$. Gaussian noise is unbounded but $\sigma=2$ keeps it close.
Complexity	$O(1)$ per paper.

F-008: Y-coordinate (Citation Height)

Field	Detail
Formula	$y = \log_{10}(1 + \text{cited_by_count}) * 15 + N(0, 1.5)$
Location	data_processor.py : compute_properties(), line ~99
Variables	cited_by_count: integer >= 0. 15: scale factor. N(0, 1.5): Gaussian noise.
Purpose	Highly cited papers float higher in the 3D scene.
Numerical Stability	$\log_{10}(1 + c) \geq 0$ for $c \geq 0$. Always produces non-negative base value.
Complexity	O(1) per paper.

F-009: Z-coordinate (Module Offset)

Field	Detail
Formula	$z = (\text{module_index} - \text{num_modules}/2) * 12 + N(0, 4)$
Location	data_processor.py : compute_properties(), line ~100
Variables	module_index: enumerated index of the paper's module. num_modules: count of unique modules. N(0,4): Gaussian noise.
Purpose	Separates disciplinary clusters along the z-axis.
Numerical Stability	No division-by-zero risk.
Complexity	O(1) per paper.

F-010: Demo Citation Distribution

Field	Detail
Formula	$\text{cited_by_count} = \text{floor}(\text{Pareto}(\alpha=1.2) * 15)$
Location	data_processor.py : generate_demo(), line ~134
Variables	alpha=1.2: Pareto shape parameter. 15: scale multiplier.
Purpose	Generates realistic heavy-tailed citation counts for demo data. A few papers get very high counts; most get low counts.
Numerical Stability	Pareto(1.2) can produce arbitrarily large values; top papers are overridden with randint(500,2000) or randint(100,500).
Complexity	O(1) per paper.

F-011: Paper ID Hash

Field	Detail
Formula	$\text{id} = \text{MD5}(\text{title} + \text{year} + \text{doi})[:12]$
Location	data_processor.py : generate_paper_id(), line ~32
Variables	title: string. year: integer (cast to string). doi: string.
Purpose	Generates a deterministic 12-character hex ID for each paper.
Numerical Stability	MD5 is deterministic. Collision probability is low for 12 hex chars (~16^12 = 2.8 * 10^14 space).
Complexity	O(len(input)) per call.

F-012: Connection Line Distance Threshold

Field	Detail
Formula	connected if <code>euclidean_distance(paper_i, paper_j) < 25</code> AND same module
Location	<code>components.py</code> : <code>three_connection_lines()</code> , JS code, line ~356
Variables	25: hardcoded distance threshold in Three.js world units.
Purpose	Draws faint lines between nearby papers in the same module cluster.
Numerical Stability	No numerical risk. Capped at $\min(2 \times \text{group_size}, 100)$ connections per module group.
Complexity	$O(k^2)$ per module group where k = group size. Total worst case $O(n^2)$.

F-013: Label Distance-Based Fade

Field	Detail
Formula	$\alpha = 0.95$ if $\text{dist} < 30$; $0.95 - (\text{dist} - 30)/100$ if $30 \leq \text{dist} < 80$; 0.12 otherwise. Clamped to min 0.08 .
Location	<code>components.py</code> : <code>three_label_system()</code> , JS code, line ~203
Variables	<code>dist</code> : Euclidean distance from camera to paper mesh. Alpha: CSS opacity $[0.08, 0.95]$.
Purpose	Labels of nearby papers are fully visible; distant labels fade to near-transparent.
Numerical Stability	Clamped to min 0.08 . No division by zero.
Complexity	$O(n)$ per frame for n labels.

2.3 Formula Dependencies

Preprocessing formulas: F-011 (ID hash, runs first during `df_to_papers`). Core computation formulas: F-001, F-002, F-003, F-004, F-005, F-006, F-007, F-008, F-009 (all run inside `compute_properties`, in sequence). F-003/F-004/F-005 depend on the sorted order produced internally. Demo-only: F-010 (only used in `generate_demo`). Visualization-only: F-012 (connection lines, runs in Three.js client-side), F-013 (label fade, runs per animation frame).

Category	Formula IDs	When Executed
Preprocessing	F-011	During <code>df_to_papers()</code> conversion
Core Computation	F-001..F-009	During <code>compute_properties()</code> , server-side, once per load
Demo Generation	F-010	Only during <code>generate_demo()</code>
Client-side Viz	F-012, F-013	Three.js render loop, per frame or on scene init

3. System Mechanism Diagram

3.1 Mermaid Diagram (Copy-Paste Ready)

```
graph TD
    subgraph Input
        A[User Uploads Files<br>CSV/JSON/XLSX/PDF/TXT/MD/DOCX/BIB]
        B[Load Demo Button]
        C[arXiv API Query]
    end

    subgraph FileParser["file_parser.py"]
        D[parse_file: dispatch by extension]
        E[Format-specific parser<br>parse_csv/json/xlsx/pdf/docx/bib/txt/md]
        F[_text_to_papers: free-text fallback]
        G[normalize_dataframe: fuzzy column matching]
    end

    subgraph DataProcessor["data_processor.py"]
        H[df_to_papers: DataFrame to Paper objects]
        I[compute_properties:<br>mass, radius, x/y/z,<br>blackhole, dyson, pagerank]
        J[compute_stats: summary statistics]
        K[generate_demo: synthetic 120 papers]
    end

    subgraph SessionState["Streamlit Session State"]
        L[papers: List of Paper]
        M[stats: dict]
        N[config: dict from config.yaml]
    end

    subgraph AIBackend["ace_model.py"]
        O[infer: fallback chain]
        P[OpenAI API]
        Q[Ollama local]
        R[GGUF llama-cpp]
        S[Fallback rules]
    end

    subgraph Frontend["Streamlit Pages + Three.js"]
        T[Home Page: metrics, top papers, module chart]
        U[Universe Page: 3D Three.js scene]
        V[Analysis Page: AI query widget]
        W[Timeline Page: year-based replay]
    end

    subgraph Components["components.py"]
        X[three_boilerplate: HTML/CSS/JS shell]
        Y[three_label_system / tooltip / orbit]
        Z[render_ai_analysis widget]
        AA[serialize/deserialize session]
    end

    A --> D --> E --> G --> H
    E --> F --> G
    B --> K --> I
    C --> H
    H --> I --> J
    I --> L
```

J --> M
L --> T
L --> U
M --> T
X --> U
Y --> U
Z --> V
V --> O --> P
O --> Q
O --> R
O --> S
AA --> L
N --> O

Render this diagram with: mermaid-cli (mmdc -i diagram.mmd -o diagram.png) or paste into mermaid.live.

3.2 Node Descriptions

Node	Description
Input (A/B/C)	Three data entry paths: file upload, demo generation, arXiv search.
FileParser (D-G)	Dispatches to format-specific parsers. Falls back to free-text extraction. Normalizes columns.
DataProcessor (H-K)	Converts DataFrames to Paper objects, computes celestial properties, generates statistics.
SessionState (L-N)	Streamlit session_state stores papers, stats, and config persistently within a browser session.
AIBackend (O-S)	infer() tries backends in order: primary -> fallbacks. Each backend wraps an LLM API.
Frontend (T-W)	Streamlit pages render metrics, Three.js scenes, AI analysis, and timeline views.
Components (X-AA)	Shared HTML/JS generators for Three.js, AI widget, session I/O.

3.3 Page-Backend Mapping

Page	Backend Modules Used	Key Components
Home (app.py)	file_parser, data_processor	File upload, metrics, bar chart, arXiv import
Universe (inferred)	data_processor, components	three_boilerplate, three_label_system, three_tooltip_code, three_orbit_controls, three_
Analysis (inferred)	ace_model, components	render_ai_analysis widget
Timeline (inferred)	data_processor, components	Year-filtered rendering (timeline_year in session state)
Settings (inferred)	ace_model	check_all_backends, API key configuration

4. Methodology

4.1 Problem Definition

Input: A heterogeneous set of academic literature files, varying in format, completeness, language, and schema. Output: A navigable 3D celestial visualization where each paper is a star, with spatial layout encoding time (x-axis), impact (y-axis), and discipline (z-axis), augmented by AI-powered natural-language analysis. Objective: Enable researchers to explore, discover patterns, and identify influential or anomalous papers in a corpus. Constraints: Must handle incomplete data gracefully. Must run on commodity hardware (no GPU required for core features). Must support offline operation (Ollama/GGUF backends). Must never crash on malformed input.

4.2 Overall Approach (Phased)

Phase	Description
Phase 1: Ingest	Universal parsing with format-specific handlers and free-text fallback. Multi-encoding support.
Phase 2: Normalize	Fuzzy column name matching maps arbitrary schemas to a standard 9-field paper schema.
Phase 3: Compute	Logarithmic mass/radius, linear time mapping, percentile-based classification, Gaussian jitter for visual spread.
Phase 4: Render	Server-side: Streamlit pages with metrics. Client-side: Three.js 3D scene with interactive controls.
Phase 5: Analyze	Multi-backend AI inference with automatic fallback chain. Per-page context injection.
Phase 6: Persist	JSON session save/load. YAML config persistence.

4.3 Key Design Decisions

- Hybrid architecture (rules + AI): The system operates fully without AI (fallback mode), ensuring the visualization always works.
- Logarithmic scaling for mass/radius: Prevents extreme outliers from dominating the visual scene while preserving relative differences.
- Fuzzy column matching: Enables the system to ingest data from Scopus, Web of Science, OpenAlex, Google Scholar, BibTeX, and arbitrary CSVs without format-specific adapters.
- Three.js r128 via iframe/HTML component: Avoids Streamlit's limited native charting. Provides full 3D interactivity.
- Fallback chain for AI: openai -> ollama -> gguf -> rule-based. Ensures the system degrades gracefully.
- Client-side label rendering: Avoids server round-trips for hover/label state changes.

4.4 Robustness Strategies

- file_parser.py: Every parser is wrapped in try/except. Final fallback treats any file as plain text. Multi-encoding tries (utf-8, utf-8-sig, latin-1, gbk, gb2312).
- normalize_dataframe: If no columns match, uses the first text column as title. If all titles are empty, generates synthetic titles.
- ace_model.py: Inference uses a fallback chain. If all backends fail, returns a rule-based response with the input word count.

- Three.js: Tooltip and label code uses try/catch around geometry parameter access.

4.5 Reproducibility and Auditability

Gaps identified: 1) No random seed is set. `compute_properties()` uses `random.gauss()` and `generate_demo()` uses `random.choices/random.randint/np.random.pareto` without seed. Recommendation: Add `random.seed(42)` and `np.random.seed(42)` at the top of `compute_properties()` and `generate_demo()`. 2) No version number is recorded in session exports. The `serialize_session()` function includes `version='4.0'` but no git hash or dependency versions. 3) No logging of inference prompts/responses for audit. `ERROR_LOG` is declared but not written to. Recommendation: Add a single-line JSONL logger in `infer()` that records timestamp, prompt hash, backend, latency, and error status.

5. Computation Methods

5.1 Data Formats and Field Definitions

Field	Type	Required	Default	Description
title	str	Yes	Untitled	Paper title (up to 300 chars)
authors	str	No	Unknown	Author names
year	int	No	2020	Publication year
cited_by_count	int	No	0	Number of citations
abstract	str	No	"	Abstract text (truncated to 500 chars)
keywords	str	No	"	Comma-separated keywords
doi	str	No	"	Digital object identifier
module	str	No	Other	Disciplinary category (ACN/ASM/ACE/CROSS/Other/CS/AI/ML/NLP/BIO/MED/PHY/MATH/EN)
journal	str	No	"	Journal or venue name

5.2 Preprocessing Steps

Step	Function	Description
P1	parse_file()	Dispatch to format-specific parser by file extension. Returns raw DataFrame.
P2	normalize_dataframe()	Fuzzy-match column names to standard schema. Apply _safe_str/_safe_int. Extract year from date strings
P3	df_to_papers()	Convert each DataFrame row to a Paper dataclass. Validate module against MODULE_COLORS; default

5.3 Core Algorithm Steps

Step	Operation	Formula Ref
S1	Compute mass = $1.0 + \log_{10}(2 + \text{cited_by_count}) * 2.5$ for each paper	F-001
S2	Compute radius = $3.0 + \log_{10}(2 + \text{cited_by_count}) * 2.5$ for each paper	F-002
S3	Compute mean_cited = arithmetic mean of all cited_by_count values	Used by F-003, F-005
S4	First-pass black hole: flag papers with $\text{cited_by_count} > \text{mean_cited} * 50$	F-003
S5	Sort papers descending by cited_by_count	F-004
S6	Assign pagerank = $1 - \text{rank_index} / n$	F-006
S7	Second-pass black hole: top 0.1% by citation	F-004
S8	Dyson sphere: top 1% AND $\text{cited_by_count} > \text{mean_cited} * 10$	F-005
S9	Compute year range [year_min, year_max]	F-007
S10	Assign module z-offsets: enumerate unique modules, space by 12 units	F-009

Step	Operation	Formula Ref
S11	For each paper: $x = \text{time_mapping} + N(0,2)$, $y = \log_citation_height + N(0,1.5)$, $z = \text{Module_offset} + N(0,4)$	

5.4 Parameter List

Parameter	Default	Adjustable	Location	Effect
warp_strength	2.0	Sidebar slider [0.5, 5.0]	config.yaml	Visual warp intensity in 3D scene
gravity_grid_segments	100	Sidebar slider [50, 200]	config.yaml	Grid resolution for gravity visualization
asteroid_particles	12000	Sidebar slider [3000, 30000]	config.yaml	Number of background particles
low_particle_mode	False	Sidebar checkbox	config.yaml	Reduces particle count for performance
fps_display	False	Sidebar checkbox	config.yaml	Show FPS counter
active_backend	openai	Sidebar selectbox	config.yaml	Primary AI backend
openai.api_key	"	Sidebar text input	config.yaml	OpenAI API key
openai.model	gpt-4o-mini	Sidebar text input	config.yaml	OpenAI model name
openai.base_url	https://api.openai.com/v1	Sidebar text input	config.yaml	OpenAI API endpoint
ollama.host	http://localhost	config.yaml	config.yaml	Ollama server host
ollama.port	11434	config.yaml	config.yaml	Ollama server port
ollama.model	llama3	Sidebar selectbox/input	config.yaml	Ollama model name
gguf.model_path	"	Sidebar file upload	config.yaml	Path to .gguf model file
gguf.n_ctx	16384	config.yaml only	config.yaml	Context window size
gguf.n_gpu_layers	-1	config.yaml only	config.yaml	GPU layers (-1 = all)
gguf.temperature	0.3	config.yaml only	config.yaml	Sampling temperature
gguf.max_tokens	4096	config.yaml only	config.yaml	Max output tokens
openai.temperature	0.3	config.yaml only	config.yaml	Sampling temperature
openai.max_tokens	4096	config.yaml only	config.yaml	Max output tokens
openai.timeout	60	config.yaml only	config.yaml	API timeout in seconds

5.5 Output Products

Product	Format	How Generated	Location
3D Universe Scene	HTML + Three.js (in iframe)	components.py functions -> st.components.html()	Rendered in browser
Dataset Statistics	Streamlit metrics + bar chart	compute_stats() -> st.metric()	Home page
Top Papers Table	Streamlit dataframe	stats['top_5'] -> st.dataframe()	Home page
AI Analysis Output	HTML div with scrollbar	render_ai_analysis() -> infer()	Analysis page / any page

Product	Format	How Generated	Location
Session JSON	JSON file download	serialize_session() -> st.download_button()	Navbar
Canvas PNG	PNG image download	EXPORT_JS -> canvas.toDataURL()	Universe page button
Config File	YAML	save_config() -> config.yaml	Project root

5.6 Complexity and Performance Bottlenecks

Operation	Complexity	Frequency	Bottleneck Risk
File parsing	$O(n)$ where n =file size	Once per upload	Large PDFs (OCR) can be slow. Mitigated by pdfplumber fallback.
normalize_dataframe	$O(n*c)$ where c =columns	Once per upload	Low risk.
compute_properties	$O(n \log n)$ due to sort	Once per load	Low risk for $n < 100k$.
Three.js scene init	$O(n)$	Once per page load	Large n (>10k) may cause slow initial render.
Connection lines	$O(k^2)$ per module group	Once per scene init	Risk: large module groups. Mitigated by cap of 100 connections/group.
Label update loop	$O(n)$	Per animation frame (60Hz)	Risk: 60k+ labels at 60fps. Mitigated by distance-based culling.
AI inference	$O(1)$ network call	Per user query	Latency depends on backend. Fallback is $O(1)$.

6. FAQ and Troubleshooting

Minimum Viable Run Path

```
# From a clean environment:
pip install streamlit pandas numpy pyyaml
streamlit run app.py
# In the browser: click 'Load Demo' button
# Expected: 120 papers loaded, metrics displayed, module chart shown
```

FAQ-1

Field	Detail
Symptom	ModuleNotFoundError: No module named 'streamlit'
Cause	Streamlit is not installed in the active Python environment.
Fix	Run: pip install streamlit pandas numpy pyyaml
Verify	Run: python -c "import streamlit; print(streamlit.__version__)" should print version.

FAQ-2

Field	Detail
Symptom	FileNotFoundError: config.yaml
Cause	config.yaml does not exist yet. This is normal on first run.
Fix	The application creates config.yaml automatically when you save settings. No action needed.
Verify	After changing any setting in the sidebar and clicking Save, config.yaml should appear in the project root.

FAQ-3

Field	Detail
Symptom	AI analysis returns '[Fallback] AI unavailable'
Cause	All configured AI backends are unreachable.
Fix	1) For OpenAI: ensure api_key is set in sidebar. 2) For Ollama: ensure Ollama is running (ollama serve). 3) For GGUF: upload
Verify	Click 'Health Check' in the sidebar. A green checkmark with backend name and latency indicates success.

FAQ-4

Field	Detail
Symptom	PDF upload returns 'No data could be extracted'
Cause	Neither pdfplumber nor pypdf is installed, or the PDF is scanned (image-only).
Fix	Run: pip install pdfplumber. For scanned PDFs, OCR is not supported in this version.
Verify	Upload a text-based PDF. The parse log should show 'Extracted N records'.

FAQ-5

Field	Detail
Symptom	DOCX upload fails
Cause	python-docx is not installed.
Fix	Run: pip install python-docx
Verify	Upload a .docx file. Parse log should show extracted records.

FAQ-6

Field	Detail
Symptom	XLSX upload fails
Cause	openpyxl is not installed.
Fix	Run: pip install openpyxl
Verify	Upload an .xlsx file.

FAQ-7

Field	Detail
Symptom	3D scene is blank / no spheres visible
Cause	papers list is empty, or the Universe page is missing from pages/ directory.
Fix	Load data first (upload or demo). Ensure the pages/ directory contains the Universe page file.
Verify	Check that the Home page shows non-zero paper count metric.

FAQ-8

Field	Detail
Symptom	Ollama connection refused
Cause	Ollama server is not running or is on a different port.
Fix	Start Ollama: ollama serve. Check host/port in sidebar config.
Verify	Run: curl http://localhost:11434/api/tags should return model list.

FAQ-9

Field	Detail
Symptom	Chinese characters display as boxes
Cause	Noto Sans SC font failed to load (no internet access).
Fix	Ensure the machine has internet access on first load, or install Noto Sans SC locally.
Verify	Switch language to Chinese. All text should render correctly.

FAQ-10

Field	Detail
Symptom	arXiv import returns 'Install arxiv package'
Cause	The arxiv Python package is not installed.
Fix	Run: pip install arxiv
Verify	Enter a query and click the arXiv Import button.

7. Getting Started Guide

7.1 Environment Setup

```
# Required: Python 3.8+
# Core dependencies (mandatory):
pip install streamlit pandas numpy pyyaml

# Optional (for full format support):
pip install pdfplumber python-docx openpyxl

# Optional (for AI backends):
pip install openai          # OpenAI backend
pip install ollama          # Ollama backend
pip install llama-cpp-python # GGUF backend (CPU or GPU)

# Optional (for arXiv import):
pip install arxiv
```

7.2 Directory Setup

```
# Ensure this structure exists:
your_project/
  app.py
  utils/
    __init__.py
    ace_model.py
    data_processor.py
    file_parser.py
    i18n.py
    theme.py
    components.py
  pages/          # Create this; add page files
  models/         # Auto-created for GGUF uploads
  logs/          # Auto-created
```

7.3 First Launch

```
streamlit run app.py
```

The browser opens to the Home page. You will see the welcome screen with the ACE-Omega logo.

7.4 Loading Data

Option A: Click the 'Load Demo' button. This generates 120 synthetic papers with realistic citation distributions. Option B: Upload one or more files using the file uploader. Supported formats: CSV, JSON, XLSX, PDF, TXT, MD, DOCX, DOC, BIB. Option C: Enter a search query in the 'arXiv/OpenAlex query' field and click 'arXiv Import'. Requires the arxiv package.

7.5 Exploring the Home Page

After loading data, the Home page displays: five metric cards (Papers, Citations, Year Range, Black Holes, Dyson Spheres), a top-5 papers table, a module distribution bar chart, and a parse log. The sidebar shows: language toggle, backend selector, visualization controls (warp, grid, particles), performance toggles, data status, model health check, and session save/load.

7.6 Configuring an AI Backend

OpenAI: Select 'OpenAI' in the sidebar. Expand 'API Config'. Enter your API key. Optionally change the model (default: gpt-4o-mini) and base URL. Click 'Save'. Click 'Health Check' to verify. Ollama: Select 'Ollama'. Ensure Ollama is running locally. The model list auto-populates. Select a model. To pull a new model, enter its name and click 'Pull'. GGUF: Select 'GGUF'. Upload a .gguf file via the file uploader. Fallback: Select 'Fallback'. No configuration needed; returns rule-based responses.

7.7 Exporting Results

Session JSON: Click 'Save Session' in the sidebar, then 'Download'. This saves all papers, positions, and settings. PNG Export: On the Universe page, click the 'Export PNG' button. This captures the Three.js canvas as a PNG image. The session JSON can be reloaded via 'Load Session' in the sidebar.

7.8 Security Notes

API keys are stored in config.yaml in plaintext. Do not commit this file to version control. Session JSON files may contain paper titles, authors, and abstracts. Treat them as potentially sensitive. The logs/ directory may contain error details. Review before sharing. No authentication is implemented. Do not expose this application to untrusted networks without adding access control.

8. Supplementary Notes

8.1 Missing Dependency Lockfile

The codebase does not include requirements.txt, Pipfile, or pyproject.toml. This is a significant gap for reproducibility. Recommended minimum requirements.txt based on code imports:

```
streamlit>=1.28.0
pandas>=1.5.0
numpy>=1.21.0
pyyaml>=6.0
# Optional:
pdfplumber>=0.9.0
pypdf>=3.0.0
python-docx>=0.8.11
openpyxl>=3.0.10
openai>=1.0.0
ollama>=0.1.0
llama-cpp-python>=0.2.0
arxiv>=2.0.0
```

8.2 Unit Test Gap

No unit tests are present. Recommended minimum test coverage targets: 1) file_parser: test each format parser with a minimal valid file and a malformed file. 2) normalize_dataframe: test with columns named differently (e.g., 'Paper Title' vs 'title'). 3) compute_properties: test with 0 papers, 1 paper, and 1000 papers. Verify mass/radius ranges. 4) infer fallback chain: mock backends to test chain behavior. A test file structure:

```
tests/
  test_file_parser.py
  test_data_processor.py
  test_ace_model.py
  test_normalize.py
  conftest.py
```

8.3 Extensibility Points

- New file format: Add a parser function in file_parser.py and register it in the PARSERS dict.
- New AI backend: Add infer_xxx() and check_xxx() functions in ace_model.py. Add the backend to the chains dict in infer().
- New visualization module color: Add to MODULE_COLORS dict in data_processor.py.
- New i18n language: Add a new key to the STRINGS dict in i18n.py.
- New Streamlit page: Create a .py file in the pages/ directory following Streamlit multi-page conventions.
- Custom celestial property formula: Modify compute_properties() in data_processor.py.

8.4 Performance Monitoring Recommendations

- 1) Add timing to parse_file() and compute_properties() via time.time() deltas, logged to st.session_state.
- 2) The Three.js scene already supports FPS display (fps_display config flag).
- 3) For large datasets (>5000 papers), consider: reducing asteroid_particles, enabling low_particle_mode, and implementing level-of-detail (LOD) for distant papers.
- 4) The connection lines algorithm ($O(k^2)$ per module group)

should be profiled for groups larger than 500 papers. Consider a spatial index (k-d tree) if performance degrades.

8.5 Configuration Reference (config.yaml)

```
# Example config.yaml structure:
model:
  active_backend: openai
  openai:
    api_key: sk-...
    base_url: https://api.openai.com/v1
    model: gpt-4o-mini
    max_tokens: 4096
    temperature: 0.3
    timeout: 60
  ollama:
    host: http://localhost
    port: 11434
    model: llama3
  gguf:
    model_path: ./models/my_model.gguf
    n_ctx: 16384
    n_gpu_layers: -1
    max_tokens: 4096
    temperature: 0.3
visualization:
  warp_strength: 2.0
  gravity_grid_segments: 100
  asteroid_particles: 12000
  low_particle_mode: false
  fps_display: false
```

Appendix A: Full Dependency List

Package	Import Location	Required	Purpose
streamlit	app.py, components.py	Yes	Web UI framework
pandas	app.py, file_parser.py, data_processor.py	Yes	Data manipulation
numpy	data_processor.py	Yes	Numerical operations
pyyaml	app.py	Yes	Config file I/O
pdfplumber	file_parser.py	No	PDF text/table extraction (primary)
pypdf	file_parser.py	No	PDF text extraction (fallback)
python-docx	file_parser.py	No	DOCX parsing
openpyxl	file_parser.py	No	XLSX parsing
openai	ace_model.py	No	OpenAI API client
ollama	ace_model.py	No	Ollama local LLM client
llama-cpp-python	ace_model.py	No	GGUF model inference
arxiv	app.py	No	arXiv API client

Appendix B: Supported File Formats

Extension	Parser Function	Notes
.csv, .tsv	parse_csv()	Multi-encoding support (utf-8, latin-1, gbk, gb2312)
.json, .jsonl	parse_json()	Handles arrays, nested objects, wrapper keys (papers, results, data, etc.)
.xlsx, .xls	parse_xlsx()	Requires openpyxl
.txt, .text	parse_txt()	Tries TSV, then free-text extraction
.md, .markdown	parse_markdown()	Extracts markdown tables, then falls back to free-text
.pdf	parse_pdf()	pdfplumber (primary), pypdf (fallback). Tables extracted if found.
.docx, .doc	parse_docx()	python-docx (primary), XML extraction (fallback for .doc)
.bib, .bibtex	parse_bib()	Regex-based BibTeX field extraction
.ris, .xml, .html, .htm	parse_txt()	Treated as structured text

Appendix C: Module Color Map

Module Code	Color Hex	Description (Inferred)
ACN	#00BFFF	Academic Citation Networks
ASM	#32CD32	Assembly / Systems
ACE	#9370DB	Academic Celestial Engine (this system)
CROSS	#FFA500	Cross-disciplinary
Other	#8ab4f8	Unclassified
CS	#00BFFF	Computer Science
AI	#9370DB	Artificial Intelligence
ML	#32CD32	Machine Learning
NLP	#FFA500	Natural Language Processing
BIO	#81c995	Biology
MED	#f28b82	Medicine
PHY	#c58af9	Physics
MATH	#fdd663	Mathematics
ENG	#ff8a65	Engineering

End of Document. Generated from source code audit of ACE-Omega Academic Universe codebase.